Ex No **Design a user login screen with user name, password, reset button and a submit button.**

Aim:-

To design an android application using Cordova for a user login screen with username, password, reset button and a submit button. Also, include header image and a label. Use layout managers.

Procedure

1. Set Up Cordova Project: Ensure Cordova is installed, then create a new project with cordova create.
2. Add Android Platform: Add the Android platform to the project using cordova platform add android.
3. Design User Interface: Create an HTML file (index.html) for the login screen layout, including input fields for username and password, along with reset and submit buttons. Utilize CSS for styling.
4. Add JavaScript Functionality: Implement JavaScript functionality (index.js) for resetting the form and handling form submission. Capture the input values and perform necessary actions (e.g., logging credentials for demo purposes).
5. Build and Run: Build the Cordova project using cordova build android, then run it on an Android device or emulator using cordova run android.
6. Test: Test the application to ensure the login screen functions as expected, allowing users to input their credentials and submit the form.

SetUp Your Cordova Project:

First, make sure you have Cordova installed. Then create a new Cordova project:

Cordova create UserLoginApp

cd UserLoginApp

Add Android Platform:

Cordova platform add android

Design the User Interface:

Create your HTML file with the login screen layout. You can use CSS for styling.

<!--index.html-->

<!DOCTYPEhtml>

<html>

<head>

<metacharset="utf-8"/>

<metaname="viewport"content="width=device-width,initial-scale=1"/>

<title>UserLogin</title>

<linkrel="stylesheet"type="text/css"href="css/index.css" />

</head>

<body>

<imgsrc="header_image.png"alt="HeaderImage"/>

<h1>Login</h1>

<formid="loginForm">

<labelfor="username">Username:</label>

<inputtype="text"id="username"name="username"/>

<labelfor="password">Password:</label>

<inputtype="password"id="password"name="password" />

buttontype="button"onclick="resetForm()">Reset</button>

<buttontype="submit">Submit</button>

</form>

<scriptsrc="js/index.js"></script>

</body>

</html>

Css:

/*css/index.css*/ body {

font-family:Arial,sans-serif;

}

. header {

text-align:center;

}

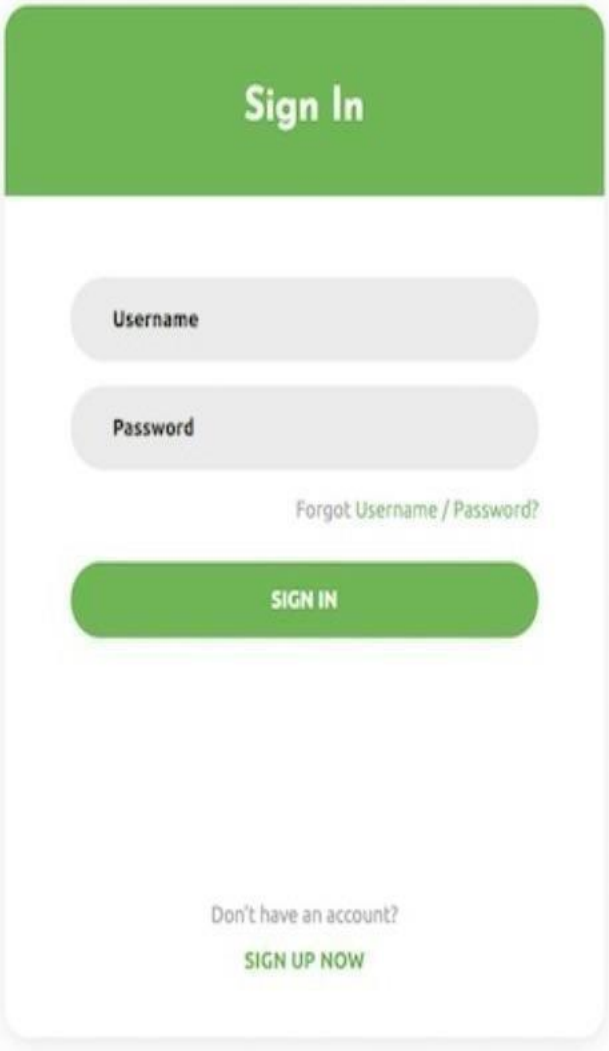
```
.container {margin:20pxauto; width: 80%;
}
label{
display: block; margin-bottom: 5px;
}
input{
width:100%; padding:8px;
margin-bottom:10px;
}
button{
padding:10px; margin-top: 10px;
}
```

Java Script Functionality:

Add JavaScript file for handling form submission and resetting.

```
//js/index.js
function resetForm() { document.getElementById("loginForm").reset();
}
document.getElementById("loginForm").addEventListener("submit",function(event)
{ event.preventDefault();//Prevent default form submission
varusername=document.getElementById("username").value;
varpassword=document.getElementById("password").value;
console.log("Username:"+username+",Password:"+password);
});
```

Output:



Result

Thus the user login screen with user name, password, reset button and a submit button was designed and executed successfully

Develop an android application using Apache Cordova to find and display the current location of the user.

Aim :-

To design and develop an android application using Apache Cordova to find and display the current location of the user.

Procedure

1. Setup Cordova Project: Create a Cordova project named "Location App" and navigate into it.
2. Add Android Platform: Use Cordova CLI to add Android platform support.
3. Install Geolocation Plugin: AddtheGeolocation plugin to enable locationservices.
4. DesignUI: Create index.html for displaying location information.
5. Implement JavaScript: Write JavaScript in index.jsp to fetch and display the user's current location.
6. Build and Run: Build the project for Android and run it on a device or emulator.
7. Test and Debug: Test the app for accurate location retrieval and handle any errors.
8. Deployment: Consider deployment options like Google Play Store for wider distribution.

//Program for android application//

Set Up Your Cordova Project:

Cordova create LocationAppcom. example.locationappLocationApp

cd LocationApp

Add the Android platform to your Cordova project:

Cordova platform add android

Install Required Plugins:

Cordova plugin add cordova -plugin- geo location

Design the User Interface:

<!--index.html-->

<!DOCTYPEhtml>

<html>

<head>

<metacharset="utf-8" />

```
<metaname="viewport"content="width=device-width,initial-scale=1"/>
<title>LocationApp</title>
<scripttype="text/javascript"src="cordova.js"></script>
</head>

<body>
<div id="locationDisplay">
<h2>CurrentLocation:</h2>
<pid="latitude">Latitude:</p>
<pid="longitude">Longitude:</p>
</div>
<scriptsrc="js/index.js"></script>
</body>

</html>
```

Java Script Functionality:

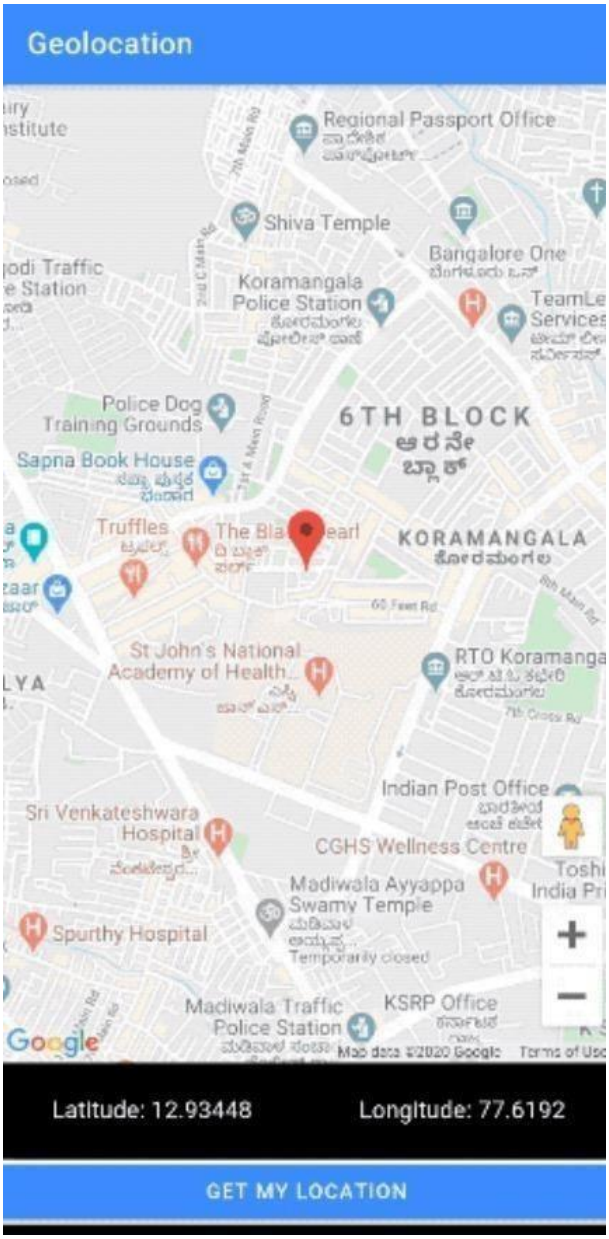
Add Java Script code to retrieve the current location using the Cordova Geolocation plugin and display it on the interface.

```
//js/index.js document.addEventListener("deviceready",onDeviceReady,false);
functiononDeviceReady()
{navigator.geolocation.getCurrentPosition(onSuccess, onError);
}

Function on Success(position){ varlatitude=position.coords.latitude;
var longitude = position.coords.longitude; document.getElementById("latitude").textContent = "Latitude: " + latitude; document.getElementById("longitude").textContent="Longitude:"+longitude;
}

functiononError(error){ alert('Erroroccurred:'+error.message);
}
```

Output:



Result

Thus the android application using Apache Cordova to find and display the current location of the user is developed executed successfully

Write programs using Java to create Android application having Databases

Aim:-

To develop an Android library application with database support for managing book availability, loans, and reservations, utilizing a centralized database server containing student information.

Procedure:

Step1.Database Schema Design: Define tables for books, students, loans, and reservations.

Step2.CreateJavaClasses: Implement classes for database operations like insertion, retrieval, and deletion.

Step3.Database Helper Class: Develop a helper class to manage database creation and versioning.

Step4.Perform Database Operations: Open a connection to the database, perform CRUD operations.

Step5.Integrate with Android UI: Link database operations with user interface elements in the Android application.

Step6.Test and Refine:Verify functionality, handle exceptions, and refine as needed for a seamless user experience.

Design Database Schema:

CREATETABLEbooks(idINTEGERPRIMARYKEY,

titleTEXT, author TEXT,

availabilityINTEGER

);

CREATETABLEstudents(idINTEGER PRIMARYKEY,

nameTEXT, email TEXT

);

CREATETABLEloans(idINTEGER PRIMARYKEY,

book_id INTEGER, student_idINTEGER, loan_date TEXT, return_date TEXT,

FOREIGNKEY(book_id)REFERENCESbooks(id),
FOREIGNKEY(student_id)REFERENCES students(id)

);

CREATETABLEreservations (id INTEGER PRIMARY KEY,

book_id INTEGER, student_id INTEGER, reservation_dateTEXT,

FOREIGNKEY(book_id)REFERENCESbooks(id),
FOREIGNKEY(student_id)REFERENCES students(id)

);

CreateJavaClassesforDatabaseOperations:

// DatabaseHelper.java publicclassDatabaseHelperextendsSQLiteOpenHelper

{

privatestaticfinalStringDATABASE_NAME= "Library.db";

privatestaticfinalintDATABASE_VERSION= 1;

publicDatabaseHelper(Contextcontext)

{super(context,DATABASE_NAME,null, DATABASE_VERSION);

}

@Override publicvoidonCreate(SQLiteDatabasedb)

{ db.execSQL("CREATE TABLE books (id INTEGERPRIMARYKEY,titleTEXT,authorTEXT,

availabilityINTEGER)");

db.execSQL("CREATETABLEstudents(id INTEGER PRIMARY KEY, name TEXT, email TEXT)")

db.execSQL("CREATETABLEloans(idINTEGER PRIMARY KEY, book_id INTEGER, student_id INTEGER, loan_date TEXT, return_date TEXT, FOREIGN KEY (book_id) REFERENCES books(id), FOREIGN KEY (student_id) REFERENCES students(id))");

db.execSQL("CREATETABLEreservations(id INTEGER PRIMARY KEY, book_id INTEGER,

student_id INTEGER, reservation_date TEXT, FOREIGNKEY(book_id)REFERENCESbooks(id), FOREIGN KEY (student_id) REFERENCES students(id))");

}

@Override

publicvoidonUpgrade(SQLiteDatabasedb,int oldVersion, int newVersion) {

//Implementif needed

}

}

PerformDatabaseOperations:

```
// Book.java publicclassBook
{ private int id;private String title; private String author; privateint availability;

//Constructor,getters,andsetters
}

// BookRepository.java publicclassBookRepository
{ privateSQLiteDatabase database;privateDatabaseHelperdbHe lper;

publicBookRepository(Contextcontext)
{ dbHelper = new DatabaseHelper(context);}
publicvoidopen(){ database=dbHelper.getWritableDatabase();
}
publicvoidclose(){ dbHelper.close();
}
publicvoidaddBook(Book book)
{ ContentValues values = new ContentValues(); values.put("title", book.getTitle());values.put("author", book.getAuthor());
values.put("availability",book.getAvailability());

database.insert("books",null,values);
}

//ImplementotherCRUDoperations
}
```

Integrate with Android UI:

```
// MainActivity.java publicclassMainActivityextendsAppCompatActivity
{privateBookRepositorybookRepository;

@Override protectedvoidonCreate(BundlesavedInstanceState){
super.onCreate(savedInstanceState); setContentView(R.layout.activity_main);
```

bookRepository=newBookRepository(this); bookRepository.open();

//Add abook

Book book = new Book(); book.setTitle("Sample Book"); book.setAuthor("SampleAuthor"); book.setAvailability(1); bookRepository.addBook(book);

}

@Override

protectedvoidonDestroy(){ super.onDestroy(); bookRepository.close();

}

}

OUTPUT:

Result

Thus the programs using Java to create Android application having Databases has been executed successfully.